GRS Project Part-A

1) Project Title: Understanding Tail Latency in Kubernetes Microservices

Kubernetes abstracts resource management, but tail latency in microservices often remains unpredictable. This project instruments Kubernetes pods using eBPF to identify kernel-level causes of p99 latency inflation (e.g., scheduling delays, socket buffering, TCP retransmissions). We will correlate pod-level SLO violations with OS- and network-level events and propose mitigation strategies.
Tech: Kubernetes + eBPF

2) Team Members

a) Rajat Kumar Srivastava i) Roll No: MT25078 ii) Mail: rajat25078@iiitd.ac.in
b) Harsha Verma i) Roll No: MT25024 ii) Mail: harsha25024@iiitd.ac.in
c) Anurag Sharma i) Roll No: MT25063 ii) Mail: anurag25063@iiitd.ac.in

3) Problem Statement

Kubernetes makes the deployment of microservices easier, but it hides low-level operating system and network behavior, and as such, tail latency (p99 latency) is hard to forecast and analyze. Conventional Kubernetes metrics don't give insight into kernel-level activities like CPU scheduling latency, socket buffering, and TCP retransmits, which can have a profound effect on application performance and cause Service Level Objective (SLO) violations. The objective of this project is to analyze tail latency in microservices running on a Kubernetes platform and apply eBPF-based observability tools to track OS and network activities at the kernel level. By correlating latency spikes and SLO violations at the pod level with kernel-level activities, this project will attempt to determine the root cause of tail latency and provide workable solutions to improve system performance.

4) Motivation

A. Importance of Tail Latency in Microservices

- Modern applications are increasingly microservice-based.
- A user request can be composed of several service calls.
- While the average latency is small the tail latency (p99) can be substantial.
- Thus tail latency is an important metric for the performance and reliability of the system.

B. Challenges in Diagnosing Latency in Kubernetes

- Kubernetes makes deployment easier.
- But it abstracts away low-level system behavior.
- Standard monitoring tools provide access to only high-level information like CPU usage, memory usage and request latency.
- These monitoring tools don't provide access to the kernel-level causes of the latency spike.

C. Impact of Kernel-Level Events

- Latency spikes in distributed systems can occur due to various operating system-related phenomena, such as: delays in CPU scheduling, delays in TCP stack processing and Resource contention
- These phenomena occur inside the Linux kernel and are hard to detect using conventional monitoring tools.

D. Role of eBPF in Observability

- It allows for the dynamic instrumentation of the Linux kernel.
- It enables probes to be attached to kernel functions and tracepoints.
- This allows for the monitoring of kernel events at a low overhead without the need to change the kernel source code.

E. Motivation for the Experiments

- The purpose of the project is to determine the impact of kernel-level behavior on the tail latency of Kubernetes microservices.
- By adding load and CPU contention, the system can measure the impact of scheduling delay and TCP processing on the latency.
- The results can help close the gap between application-level latency metrics and kernel-level system behavior.

5) Objectives

A. Deploy Kubernetes Microservice

- Create a single node Kubernetes cluster using Minikube.
- Deploy a microservice based on NGINX using a Kubernetes Deployment object.
- Expose the microservice so it can be accessed reliably within the cluster by creating a Kubernetes Service object for it.
- Use these two objects as a base environment for all latency experiments.

B. Instrument Kernel Network Behavior

- Utilize eBPF-based tracing through the use of the bpftrace tool in order to observe kernel-level network behavior.

- Utilize the probes for the tcp_sendmsg function in order to measure the time spent within the TCP send path.
- Latency value histograms are utilized in order to measure the kernel-level time spent within the TCP processing path.

C. Observe CPU Scheduling Delay

- Instrument Linux scheduler events using eBPF tracepoints.
- Monitor events such as sched_wakeup and sched_switch to estimate the run queue waiting time.
- Estimate the time taken by the task before it is scheduled on the CPU.
- These metrics can be used to measure the effect of the OS scheduler on the latency of the requests.

D. Generate Load and Contention

- Use continuous curl requests to create application traffic.
- Use wrk as a tool to benchmark the application.
- Make the requests concurrent to increase the pressure on the microservice.
- Use a stress workload to introduce CPU contention.
- Use normal and stressed conditions to test the system.

E. Study Tail Latency Behaviour

- Latency measurement under baseline, stressed, and parallel load scenarios.
- Observing the distribution of request latency through p50, p95, and p99 metrics.
- Comparing the effects of CPU contentions and concurrency on higher percentiles of latency.
- Plotting the results to illustrate the increase in tail latency during unfavorable scenarios.
- Correlation between application-level latency spikes and kernel-level events using eBPF.

6) Background

A. Kubernetes

- Kubernetes is a container orchestration platform for deployment and management of containerized applications.
- Applications are deployed as containers within a pod, and a pod is the smallest unit of deployment in Kubernetes.
- Kubernetes services are network endpoints for accessing applications in a stable manner.

- Kubernetes is a platform for container orchestration, scaling, and management of containers.

B. Tail Latency

- Latency is defined as the time it takes for a request to be serviced and returned to the client.
- Performance metrics are usually quantified using percentiles such as p50, p95, and p99.
- Latency at p50 represents the median latency for a system, whereas latency at p99 represents the slowest 1% of the system's latency.
- The importance of tail latency is significant since it can affect system performance in distributed applications.

C. eBPF

- eBPF (extended Berkeley Packet Filter) is a technology that allows for the safe execution of custom programs within the Linux kernel.
- It allows developers to instrument kernel functions and trace system events without changing the kernel source code.
- eBPF programs can be attached to kernel probes, tracepoints, and network hooks for observing system behavior.
- Tools like bpftrace can be used for dynamic tracing of kernel functions for performance analysis.

D. Cilium & Hubble

- Cilium is a networking solution provided by Kubernetes that utilizes the eBPF-based datapath technology for networking and security purposes.
- Cilium replaces the traditional networking stack with eBPF programs in the kernel.
- Hubble is the observability part of Cilium that provides observability for the network flows between pods.
- With the help of Hubble, users can monitor the communication and events within the Kubernetes cluster.

7) Methodology/Design

System Design

- The system is a microservice-based system built with Kubernetes and kernel-based observability tools.
- A client workload sends HTTP requests to a microservice running in a Kubernetes system.
- HTTP requests are serviced via a Kubernetes Service and are proxied to an nginx pod running in the system.

- Traffic flows through the Cilium eBPF dataplane and the Linux TCP network stack.
- Kernel-based observability tools are attached using the eBPF technology and observe system events related to latency.

Pipeline used in the system

Load Generator
↓
Kubernetes Service
↓
nginx Pod
↓
Cilium eBPF Dataplane
↓
Linux TCP Stack
↓
eBPF Probes

Components of the System

A. Kubernetes Cluster

- A single node Kubernetes cluster is deployed using Minikube.
- Minikube is used as an environment to deploy and test containerized applications.
- The cluster is responsible for scheduling pods, networking, and service discovery.
- Kubernetes commands, abbreviated as kubectl, can be used to deploy and monitor the state of the system.

B. Microservice Deployment

- A simple web server using nginx is used as the microservice application.
- The microservice application is deployed by using a Kubernetes Deployment object.
- Kubernetes Service is used to expose the microservice application, providing a stable endpoint to receive incoming requests.
- The Kubernetes Service routes incoming traffic to the nginx pod by using selectors.

C. Traffic Generation

- Application load is created to mimic real client requests.
- Two traffic generation tools are used:

Continuous Curl Requests

- A container for curl is created to send HTTP requests to the nginx service.
- This generates traffic within the Kubernetes cluster.

Benchmark Traffic using wrk

- The wrk tool is used to benchmark the service by sending concurrent requests to the service.
- This is to test the microservice with varying concurrency of requests.

D. Kernel-Level Instrumentation

- Kernel events are tracked by eBPF tracing tools.
- eBPF programs are dynamically attached to kernel function calls.

TCP Send Latency Probe

- The probe is attached to the kernel function call for tcp_sendmsg.
- The kernel function call is responsible for sending data from the application to the network stack using TCP.
- The probe tracks the latency for the TCP send path.
- The latency values are tracked and summarized in histograms.

CPU Scheduling Latency Probe

- The scheduler is tracked by tracepoints.
- The events are sched_wakeup and sched_switch.
- The events are tracked to estimate how long a task waits in the CPU run queue before execution.
- The events provide insight into scheduling delays, which may contribute to latency.

E. CPU Stress Experiment

- The contention for the CPU is achieved by using the "polinux/stress" container.
- The stress workload has multiple CPU-bound tasks.
- This increases the contention for the CPU resources on the node.
- The experiment can be used to understand the impact of CPU scheduling delays on the microservice's tail latency.

8) Initial Implementation

A. Cluster Setup

- The Kubernetes environment is created using Minikube, which is a tool for running a single-node Kubernetes cluster locally.
- The script 9_start_minikube.sh is used to start the Kubernetes environment.
- The script 9_install_cilium.sh is used to install Cilium, which is a network solution for Kubernetes based on eBPF.
- Cilium replaces the default network layer and allows for kernel-level observability.

- The installation of Cilium also allows for Hubble, which is used for network flow visibility between pods.

B. Microservice Deployment

- A simple web server based on the nginx web server was used as a microservice application.
- The deployment configuration was specified in the 9_nginx_deployment.yaml file, which includes the container image and pod configurations.
- A Kubernetes Service was specified in the 9_nginx_service.yaml file, which was used to expose the nginx pod.
- A script, 9_deploy_nginx.sh, was used to apply the configurations specified in the above files to the cluster, thereby deploying the application.
- This service was used as a target for the traffic generated during the experiment.

C. Kernel Probes

- Kernel-level measurements were collected through the bpftrace tool.
- The script 9_baseline_bpftrace.sh is used to attach the probe to the kernel function tcp_sendmsg.
- This probe is used to measure the time taken inside the TCP send path for outgoing packets.
- The script 9_cpu_scheduling.sh is used to measure scheduler-related events like sched_wakeup and sched_switch.
- These measurements can be used to estimate the waiting time of tasks in the CPU run queue.

D. Load Experiments

- Traffic was generated to simulate client requests to the microservice.
- The script 9_run_latency_script.sh generates HTTP load and measures latency statistics.
- This allows the system to observe request latency under different levels of concurrency.
- CPU contention was introduced using the script 9_cpu_stress.sh, which runs a stress workload.
- The stress workload creates multiple CPU-bound tasks that compete for CPU resources.

E. Monitoring and Observability

- Network flows inside the Kubernetes cluster were monitored using Hubble.
- The command hubble observe provides real-time visibility into communication between pods.

- This helps confirm that traffic is correctly flowing from the load generator to the nginx service.
- Combined with kernel probes, this provides both network-level and kernel-level observability.

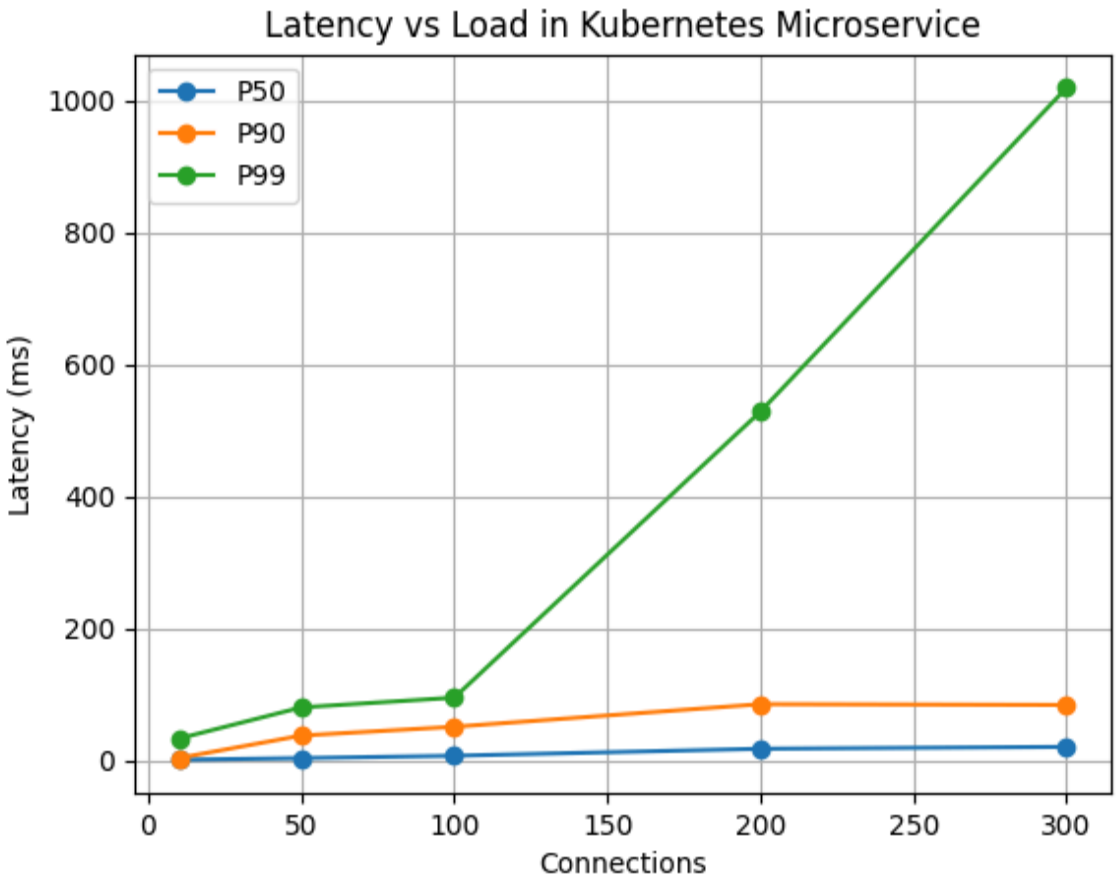
Experimental Setup using wrk at application layer

- The nginx microservice was deployed in a Minikube cluster.
- HTTP load was generated using the wrk benchmarking tool.
- Multiple connection levels were tested by 9_run_latency_script.sh
- Latency metrics p50, p90, and p99 were collected.
- The experiment aims to measure the effect of increased request load on latency distribution.

8) Evaluation & observation

Latency vs Load Experiment using wrk at application layer

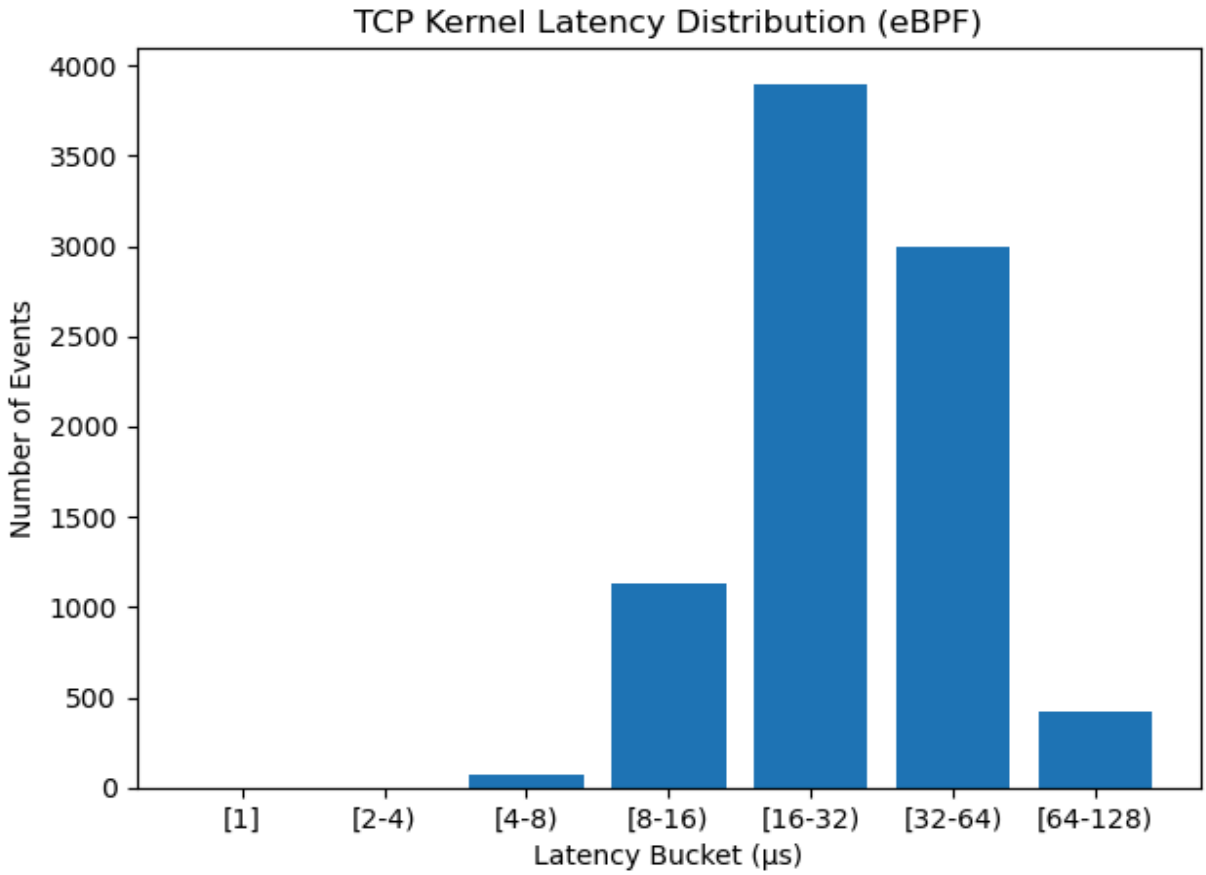
Connections	P50(ms)	P90(ms)	P99(ms)
10	1.01	3.69	33.19
50	3.58	37.74	80.46
100	7.13	51.11	95.28
200	17.73	85.34	525.99
300	20.62	84.21	1020



Observation

- It is observed that as the number of connections increases, the latency of the request also increases. This implies that when the load on the microservice increases, the average latency of the microservice also increases.
- It is observed that median latency, or P50, increases gradually as the number of connections increases. The median latency increases from 1 ms at 10 connections to 20 ms at 300 connections.
- Percentile values such as P90 and P99 increase at a much faster rate than median latency, or P50.
- The latency of P99 increases sharply after 200 connections, showing high values of over 1000 ms at 300 connections.
- These results show that although the average performance of microservices is satisfactory, some of the requests are handled at a slower rate when the number of connections increases.

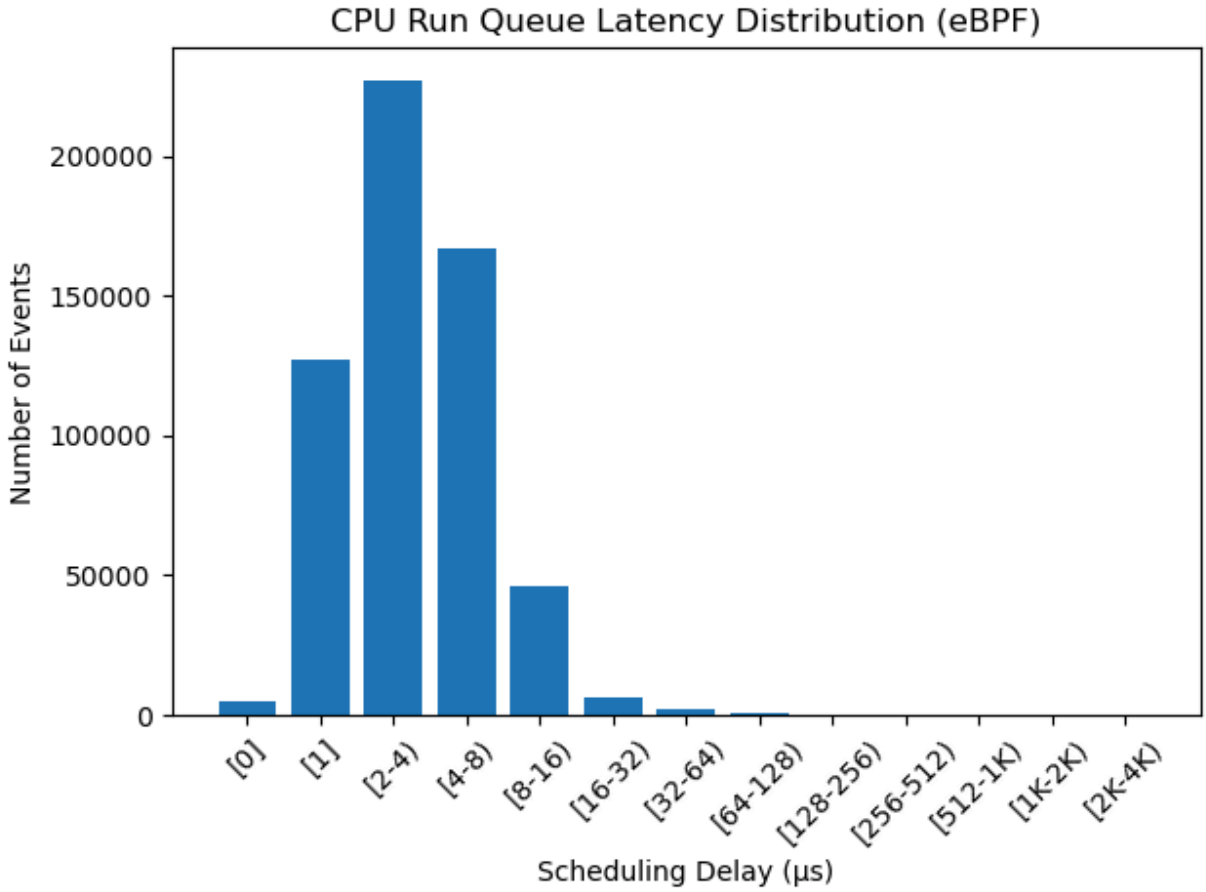
TCP Send Latency Probe



Observation

Although the average latency of the TCP send path operations is low, the existence of operations at a higher latency level indicates kernel-level variability, which might result in increased p99 latency in microservices.

CPU Scheduling Latency Probe



Observation

Tail events reaching 1–4 ms indicate that tasks may occasionally wait longer for CPU scheduling.Such delays can propagate to application-level latency, contributing to p99 response time inflation in Kubernetes workloads.

Outcomes & Conclusion

Key Outcomes

- The project successfully implemented a microservice environment using Kubernetes, Minikube, and nginx.
- The instrumentation at the kernel level was successfully implemented using eBPF probes with bpftrace.
- The experiments showed that CPU contention can cause delays in scheduling, thereby increasing request latency.
- Increased concurrency in requests also led to greater variability in request latency.
- The use of eBPF tracing also provided insight into the kernel level, which is typically invisible behind the abstractions in Kubernetes.

- The use of Cilium/Hubble network monitoring in tandem with kernel-level tools also allowed visibility into network and OS activities.

Conclusion

- System-level factors such as CPU scheduling and network processing can also be relevant in microservices architecture when it comes to tail latency.
- Traditional monitoring tools are generally limited to providing application-level metrics.
- eBPF-based instrumentation allows for detailed insights into kernel behavior at a low overhead.
- The implemented system aims to show the benefits of cross-layer observability in determining the possible causes of the increase in latency.

Future Work

- Expand the experiments to include multi-node Kubernetes environments.
- Instrument other kernel events of interest, such as TCP retransmissions and network buffering.
- Experiment with larger and more complex microservice workloads.
- Experiment with different resource limits and scheduling policies to observe their effect on latency.

10) References

- Kubernetes Documentation https://kubernetes.io/docs
- eBPF Documentation https://ebpf.io
- Cilium Documentation https://docs.cilium.io
- bpftrace Documentation https://github.com/iovisor/bpftrace
- Minikube Documentation https://minikube.sigs.k8s.io/docs